What Is Git?

Git is a version control system. It tracks changes to files over time, letting you save snapshots of your work, review your history, revert to earlier versions, and collaborate with others without overwriting each other's contributions.

For math majors, Git solves a problem that shows up constantly in upper-level coursework: you revise a proof, realize the earlier version was closer to correct, and have no way to get it back. Or you work on a problem set across multiple machines and end up with conflicting copies. Git eliminates both problems. Every change is recorded, every version is recoverable, and your work is stored in one authoritative place.

This guide focuses on using Git from the command line. The command line gives you full control and is what you will encounter in any professional or research context. Once you understand the fundamentals here, graphical Git tools will make immediate sense.

Table of Contents

1. Core Concepts
2. Setting Up Git
3. Creating a Repository
4. The Basic Workflow
5. Viewing History
6. Undoing Changes
7. Branching
8. Working with GitHub
9. Cloning a Repository
10. Common Workflows for Math Majors
11. The .gitignore File

---

1. Core Concepts

Before running any commands, understand the vocabulary Git uses. These terms appear everywhere in the documentation and in error messages.

Repository (repo) A repository is a folder that Git is tracking. It contains your files and a hidden `.git` folder where Git stores the entire history of your project. You will have one repository per project (one for your LaTeX notes, one for your R scripts, and so on).

Commit A commit is a saved snapshot of your repository at a specific point in time. Think of it as a checkpoint. Every commit has a unique ID and a message you write describing what changed. Your project history is a sequence of commits.

Staging area (index) Before you commit, you choose which changes to include using a staging area. This lets you commit specific files or specific changes rather than everything at once. The staging area sits between your working files and your commit history.

Branch A branch is an independent line of development within a repository. By default, your repository has one branch called `main`. Branches let you try something new (a different approach to a proof, a restructured script) without touching the stable version of your work.

Remote A remote is a version of your repository stored somewhere else, typically on GitHub. You push changes to the remote to back up your work and pull changes from it to sync updates.

Working directory The working directory is simply the folder on your computer containing your actual files. When you edit a file, you are editing in the working

directory. Those changes are not saved to Git history until you stage and commit them.

The relationship between these pieces:

```
Working Directory ⟶ Staging Area ⟶ Local Repository ⟶ Remote
(GitHub)
     (edit files)       (git add)         (git commit)       (git
push)
```

---

2. Setting Up Git

If you have not yet installed Git, refer to the Foundations section of this guide. Once Git is installed, complete the following configuration steps before doing anything else.

Set your name and email:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Use the same email address associated with your GitHub account. This information is embedded in every commit you make.

Set your default branch name to `main`:

```
git config --global init.defaultBranch main
```

Older versions of Git use `master` as the default branch name. This command standardizes it to `main`, which matches GitHub's current default.

Set your default text editor:

When Git needs you to write a message (such as during a merge), it opens a text editor. Set this to one you are comfortable with.

For VS Code:

```
git config --global core.editor "code --wait"
```

For Nano (simple terminal editor, good default if you are unsure):

```
git config --global core.editor "nano"
```

Confirm your configuration:

```
git config --list
```

This prints all current Git configuration values. Verify your name and email appear correctly.

---

3. Creating a Repository

There are two ways to get a Git repository: initialize one in an existing folder, or clone one that already exists on GitHub. This section covers initialization. Cloning is covered in Section 9.

Navigate to your project folder:

```
cd path/to/your/project
```

For example, if you have a folder called `real-analysis` in your Documents:

```
cd ~/Documents/real-analysis
```

Initialize the repository:

```
git init
```

Git creates a hidden `.git` folder inside your project folder. Your project is now a Git repository. No files are tracked yet.

Check the status of your repository:

```
git status
```

`git status` is the most useful command in Git. It tells you which files are tracked, which have changes, and what is staged. Run it constantly, especially when learning.

At this point, Git will report that you have untracked files (all of them, since nothing has been committed yet).

---

4. The Basic Workflow

Every time you work on a project, you follow the same cycle: edit files, stage the changes you want to save, and commit them with a message.

Step 1: Edit your files

Work normally. Write your LaTeX proof, update your R script, add a new Python function. Git is watching the folder but not recording anything yet.

Step 2: Check what changed

```
git status
```

Git lists files that have been modified or created since your last commit. Modified files appear under "Changes not staged for commit." New files that Git has never seen appear under "Untracked files."

Step 3: Stage your changes

To stage a specific file:

```
git add filename.tex
```

To stage all changed and new files at once:

```
git add .
```

The period (`.`) means "everything in the current directory and below." Use it carefully: make sure you are not accidentally staging files you did not intend to include. Running `git status` before and after `git add` is a good habit.

To stage only part of a file (specific changes within a file):

```
git add -p filename.tex
```

This enters an interactive mode that steps through each changed section and asks whether to stage it. Useful when a file has multiple unrelated changes.

Step 4: Commit the staged changes

```
git commit -m "Your commit message here"
```

The `-m` flag lets you write the message directly in the command. Keep messages short and specific. Describe what changed and why, not just what files you touched.

Good commit messages:

```
Add definition of uniform continuity
Fix indexing error in matrix multiplication script
Reorganize analysis notes into theorem and proof structure
```

Bad commit messages:

```
update
stuff
fix
changes
```

Your commit messages become the readable history of your project. A year from now, a good message tells you exactly why a change was made.

Step 5: Repeat

Continue editing, staging, and committing. There is no rule about how often to commit. A reasonable guideline: commit whenever you finish a logical unit of work. Finishing a proof, completing a function, fixing a specific bug. More frequent commits give you more checkpoints to return to.

---

5. Viewing History

View the commit log:

```
git log
```

This prints the full history of commits, from most recent to oldest. Each entry shows the commit ID (a long hash), the author, the date, and the message.

View a condensed log:

```
git log --oneline
```

Each commit is shown on a single line with a shortened ID and the message. Much easier to scan.

View the log as a graph (useful when working with branches):

```
git log --oneline --graph --all
```

See what changed in a specific commit:

```
git show <commit-id>
```

Replace `<commit-id>` with the short or full hash from `git log`. Git shows the commit metadata and a diff of every change made in that commit.

See what changed between your working directory and the last commit:

```
git diff
```

This shows unstaged changes. To see staged changes (what is about to be committed):

```
git diff --staged
```

---

6. Undoing Changes

Git's ability to undo changes is one of its most important features. The right command depends on what you want to undo and how far back you want to go.

Discard changes to a file before staging (restore to last commit):

```
git restore filename.tex
```

This throws away all changes to that file since the last commit. The changes are gone permanently. Use with care.

Unstage a file (remove from staging area without discarding changes):

```
git restore --staged filename.tex
```

The file goes back to the working directory with its changes intact. Nothing is deleted.

Undo the last commit but keep the changes staged:

```
git reset --soft HEAD~1
```

`HEAD~1` means "one commit before the current one." The commit is removed from history, but everything that was in it stays staged, ready to recommit. Useful when you committed too early or with the wrong message.

Undo the last commit and unstage the changes:

```
git reset HEAD~1
```

The commit is removed and the changes go back to the working directory, unstaged. The files themselves are untouched.

Undo the last commit and discard all changes permanently:

```
git reset --hard HEAD~1
```

This is destructive. The commit is removed and the changes are deleted from your working directory. Only use this when you are certain you do not need those changes.

Create a new commit that reverses an old commit (safe for shared history):

```
git revert <commit-id>
```

Rather than rewriting history, `git revert` creates a new commit that undoes the changes from the specified commit. This is the safest way to undo something that has already been pushed to GitHub.

---

7. Branching

Branches let you work on something new without touching the stable state of your project. In a math context, this is useful when you want to try a different approach to a proof or restructure your notes without risking the version that is currently working.

List all branches:

```
git branch
```

The current branch is marked with an asterisk.

Create a new branch:

```
git branch branch-name
```

Switch to a branch:

```
git switch branch-name
```

Create a branch and switch to it in one step:

```
git switch -c branch-name
```

This is the most common pattern. You create and move to the new branch immediately.

Merge a branch into your current branch:

When your work on a branch is complete and you want to bring it into `main`:

```
git switch main
git merge branch-name
```

If there are no conflicts, Git merges automatically and creates a merge commit. If both branches changed the same part of the same file, Git reports a conflict and asks you to resolve it manually.

Resolving a merge conflict:

Open the conflicting file. Git marks the conflicting sections:

```
<<<<<<< HEAD
Content from your current branch
=======
Content from the branch being merged
>>>>>>> branch-name
```

Edit the file to keep what you want (delete the markers and the unwanted content), then stage and commit:

```
git add filename.tex
git commit -m "Resolve merge conflict in filename"
```

Delete a branch after merging:

```
git branch -d branch-name
```

---

8. Working with GitHub

GitHub is a hosting service for Git repositories. It stores a remote copy of your repository online, which serves as both a backup and a way to access your work from any machine.

Create a repository on GitHub:

1. Go to https://github.com and sign in
2. Click the **+** icon in the top right and select **New repository**
3. Name the repository (use the same name as your local project folder for clarity)
4. Leave it set to **Public** or choose **Private** depending on your preference
5. Do not initialize with a README if you already have a local repository
6. Click **Create repository**

GitHub gives you a URL for the new repository, in the format:

```
https://github.com/yourusername/your-repo-name.git
```

Connect your local repository to GitHub:

```
git remote add origin https://github.com/yourusername/your-repo-name.git
```

`origin` is the conventional name for your primary remote. You can call it anything, but `origin` is the standard.

Verify the remote was added:

```
git remote -v
```

This prints the fetch and push URLs for each remote.

Push your commits to GitHub:

```
git push -u origin main
```

The `-u` flag sets `origin main` as the default upstream, so future pushes can be run with just:

```
git push
```

Pull changes from GitHub:

If you made changes on another machine or directly on GitHub (such as editing a file through the browser), pull them to your local repository:

```
git pull
```

This fetches and merges the remote changes into your current branch.

Authenticating with GitHub:

GitHub no longer accepts your account password for command-line operations. You have two options:

Option 1 - Personal Access Token (simpler):

1. Go to GitHub Settings > Developer Settings > Personal Access Tokens > Tokens (classic)
2. Generate a new token with `repo` scope
3. When Git asks for your password in the terminal, paste the token instead

Option 2 - SSH key (more convenient for frequent use):

1. Generate a key pair: `ssh-keygen -t ed25519 -C "you@example.com"`
2. Copy the public key: `cat ~/.ssh/id_ed25519.pub`
3. Add it to GitHub under Settings > SSH and GPG Keys
4. Use the SSH remote URL instead: `git@github.com:yourusername/your-repo-name.git`

SSH is the better long-term setup because you stop being prompted for credentials on every push.

---

9. Cloning a Repository

Cloning downloads an existing repository from GitHub (or any remote) to your local machine, including the full commit history.

Clone a repository:

```
git clone https://github.com/username/repository-name.git
```

This creates a new folder with the repository name in your current directory, downloads all files and history, and automatically sets up the remote connection to `origin`.

To clone into a folder with a different name:

```
git clone https://github.com/username/repository-name.git my-folder-name
```

After cloning, navigate into the folder and work normally:

```
cd repository-name
```

```
git status
```

---

10. Common Workflows for Math Majors

Workflow: Tracking LaTeX files for a course

Start at the beginning of the semester:

```
mkdir math-401
cd math-401
git init
git remote add origin https://github.com/yourusername/math-401.git
```

Create your first LaTeX file and make an initial commit:

```
git add notes.tex
git commit -m "Initial notes file"
git push -u origin main
```

After each study session, commit and push:

```
git add .
git commit -m "Add proof of Heine-Cantor theorem"
git push
```

Workflow: Backing up your Obsidian vault

Your vault is just a folder of Markdown files, which Git handles perfectly.

```
cd ~/Documents/Math-Notes
git init
```

```
git remote add origin https://github.com/yourusername/math-notes.git
git add .
git commit -m "Initial vault backup"
git push -u origin main
```

After each session, run the three-command backup:

```
git add .
git commit -m "Session notes: real analysis, lecture 12"
git push
```

Workflow: Experimenting with a Python or R script

When you want to try a significant change to an existing script without breaking the working version:

```
git switch -c experiment-log-scale
```

Make your changes on this branch. If they work, merge them back:

```
git switch main
git merge experiment-log-scale
git branch -d experiment-log-scale
```

If they do not work, delete the branch without touching `main`:

```
git switch main
git branch -D experiment-log-scale
```

---

11. The .gitignore File

Not everything in your project folder should be tracked by Git. Compiled files, temporary outputs, system files, and large data files add noise to your history and sometimes contain information you do not want stored publicly.

A `.gitignore` file tells Git which files and folders to ignore. Create it in the root of your repository:

```
touch .gitignore
```

Open it in any text editor and add patterns for files to ignore. Below is a recommended starting `.gitignore` for math majors using LaTeX, Python, and R:

```
# LaTeX auxiliary files
*.aux
*.log
*.out
*.toc
*.fls
*.fdb_latexmk
*.synctex.gz

# Python
__pycache__/
*.pyc
*.pyo
.ipynb_checkpoints/

# R
.Rhistory
.RData
.Rproj.user/

# macOS system files
.DS_Store

# Windows system files
Thumbs.db
```

```
# Large data files (add specific extensions as needed)
*.csv
*.xlsx
```

Add and commit the `.gitignore` file itself:

```
git add .gitignore
git commit -m "Add .gitignore"
```

If Git is already tracking a file you want to ignore:

Adding it to `.gitignore` will not stop Git from tracking it once it is in the history. You need to untrack it explicitly:

```
git rm --cached filename
git commit -m "Stop tracking filename"
```

The file stays in your working directory but is no longer tracked by Git.

---

12. Quick Reference

Task	Command
Initialize a repository	`git init`
Check status	`git status`
Stage a file	`git add filename`
Stage all changes	`git add .`
Commit staged changes	`git commit -m "message"`
View commit history	`git log --oneline`
See unstaged changes	`git diff`

Task	Command
See staged changes	`git diff --staged`
Discard file changes	`git restore filename`
Unstage a file	`git restore --staged filename`
Undo last commit (keep changes)	`git reset HEAD~1`
Add a remote	`git remote add origin <url>`
Push to GitHub	`git push`
Pull from GitHub	`git pull`
Clone a repository	`git clone <url>`
Create a branch	`git switch -c branch-name`
Switch branches	`git switch branch-name`
Merge a branch	`git merge branch-name`
Delete a branch	`git branch -d branch-name`
View all branches	`git branch`
View remotes	`git remote -v`